



DC – AI – Routing – Switching – Security – Wireless – Automation

vJunos Deployment on KVM

An HPE Juniper Networking Techpost

Ridha Hamidi

2023-05-10

HPE



Contents

1	vJunos Deployment on KVM	3
1.1	Summary	3
1.2	Introduction	3
1.3	Prepare the Environment	3
1.3.1	Update packages	4
1.3.2	Check if the Server Supports Hardware Virtualization	4
1.3.3	Check if VT is Enabled in the BIOS	4
1.3.4	Install KVM	4
1.3.5	Verify installation	4
1.4	Deploy vJunos-switch	5
1.4.1	Pre-requisite Tips	5
1.4.2	Download vJunos Images	5
1.4.3	Deploy vJunos-switch Instances	6
1.4.4	Deploy vJunos	9
1.4.5	Verifications	11
1.5	Configuration	13
1.5.1	Base vJunos-switch Configuration	13
1.6	Adding Apstra Controller	15
1.7	Conclusion	16
1.8	Useful links	17
1.9	Glossary	17
1.10	Acknowledgements	17

List of Figures

1 Network Topology 6

1 vJunos Deployment on KVM

Ridha Hamidi - 05/10/2023

A comprehensive user guide on how to successfully deploy and use vJunos-switch and vJunosEvolved on KVM (one of the most popular virtualized environments in the community, alongside EVE-NG and GNS3).

1.1 Summary

Juniper is releasing a new virtual test product named vJunos that is targeted at data center and campus switching use cases.

vJunos comes in two flavours for the initial release: - vJunos-switch : based on the legacy Junos OS running on Free BSD, and targeted for data center and campus switching use cases - vJunosEvolved based on the newer Junos OS Evolved on top of Linux, and targeted for both routing and switching use cases

This post provides a comprehensive user guide on how to successfully deploy and use vJunos-switch and vJunosEvolved on KVM. This other post explains [how to deploy vJunos-switch and vJunosEvolved on EVE-NG](#).

1.2 Introduction

The steps used on our setup and detailed in the rest of this post are:

- Prepare their KVM environment for vJunos deployment
- Deploy vJunos
- Troubleshoot some of the most common deployment issues
- Build a simple EVPN-VXLAN topology using multiple vJunos instances managed by Juniper Apstra
- Verify your work

We will try to provide comprehensive explanations about the procedure and explain all the steps. However, for the sake of brevity, we will not address a few topics that might be of interest for some users, like using vJunos with ZTP. This can be the topic of a separate post in the future.

Let's get started.

1.3 Prepare the Environment

The server we will use in our deployment has the following:

- Server: Supermicro SYS-220BT-HNC9R
- CPUs: 128 x Intel(R) Xeon(R) Gold 6338 CPU @ 2.00GHz
- RAM: 256 GB DDR4
- SSD: 1 TB
- OS: Ubuntu 20.04.5 LTS

We will first verify that this server supports virtualization, then we will install KVM components.

1.3.1 Update packages

```
1 user@host:~$ sudo apt-get update && sudo apt-get upgrade -y
2
3 <output truncated>
```

1.3.2 Check if the Server Supports Hardware Virtualization

```
1 user@host:~$ grep -Eoc '(vmx|svm)' /proc/cpuinfo
2 128
3 user@host:~$
```

In our case, our server has 128 CPUs that have “vmx” or “svm” flags enabled to support virtualization

1.3.3 Check if VT is Enabled in the BIOS

This is done by installing and using the “kvm-ok” tool, which is included in the cpu-checker package.

```
1 user@host:~$ sudo apt-get install cpu-checker -y
2 <output truncated>
3
4 user@host:~$ kvm-ok
5 INFO: /dev/kvm exists
6 KVM acceleration can be used
7 user@host:~$
```

Now that we have checked that the server is ready, we can proceed and install KVM.

1.3.4 Install KVM

There are several packages that need to be installed, some are mandatory and some are optional

- qemu-kvm: software that provides hardware emulation for the KVM hypervisor.
- libvirt-bin: software for managing virtualization platforms.
- bridge-utils: a set of command-line tools for configuring ethernet bridges.
- virtinst : a set of command-line tools for creating virtual machines.
- virt-manager: provides an easy-to-use GUI interface and supporting command-line utilities for managing virtual machines through libvirt.

We included all mandatory and optional packages in the same install command

```
1 user@host:~$ sudo apt-get install qemu-kvm bridge-utils virtinst virt-manager -y
2
3 <output truncated>
```

1.3.5 Verify installation

```
1 user@host:~$ sudo systemctl is-active libvirtd
2 active
3 user@host:~$ virsh version
4 Compiled against library: libvirt 4.0.0
5 Using library: libvirt 4.0.0
6 Using API: QEMU 4.0.0
7 Running hypervisor: QEMU 2.11.1
8
9 user@host:~$ apt show qemu-system-x86
```

```
10 Package: qemu-system-x86
11 Version: 1:2.11+dfsg-1ubuntu7.41
12 Priority: optional
13 Section: misc
14 Source: qemu
15 Origin: Ubuntu
16 <output truncated>
```

Now that KVM is installed and up and running, we can proceed and deploy vJunos.

1.4 Deploy vJunos-switch

1.4.1 Pre-requisite Tips

There are a few pieces of information to know about Juniper vJunos before you start building your virtual lab. These are the most important ones:

- vJunos is not an official Juniper Networks product, it is a test product, so it is neither sold nor officially supported by Juniper's TAC. If you run into any issues, we encourage you to report it in the Juniper community. Juniper Networks makes vJunos available for free to download and use with no official support.
- It is recommended to use vJunos for feature testing only, and not for any scaling or performance tests. For the same reasons, it is not recommended to use vJunos in production environments. It is recommended to join the [Juniper Labs Community](#) if you need further support on vJunos
- A vJunos instance is composed of one single VM that nests both Control and Data Planes, hence some limitations of deploying options of vJunos. Please see vJunos FAQ.
- Default login of vJunos is root with no password.
- vJunos must be provisioned with at least one vNIC for the management, and as many vNICs as there are data plane interfaces.
- It is our experience that vNICs are more reliable if you configure them to use virtio driver. We have not experienced any issue when using this driver, instead of the other ones, like e1000.
- There is no license to use vJunos, so all features should work without entering any license key, even though some features will trigger warnings about missing licenses. You can ignore those warning and move on.
- You need to have an account with Juniper support to have access to the download page. It is recommended to select "Evaluation User Access" that gives you access to evaluation software. If you are a new user, beware that creating a new user account is not immediate, as it goes through an approval process that might take up to 24 hours, so plan accordingly.

Now that all these notes are read and well understood, we're ready to deploy our first vJunos, knowing all limitations and restrictions.

1.4.2 Download vJunos Images

vJunos-switch image can be downloaded from the official Juniper support page <https://support.juniper.net/support/downloads/?p=vjunos>.

vJunosEvolved image can be downloaded from the official Juniper support page <https://support.juniper.net/support/downloads/?p=vjunos-evolved>

At the time of writing this post, the latest vJunos release is 23.1R1.8.

Note that in addition to vJunos images, you will also need a Linux image to deploy host VMs to be connected to vJunos instances for testing purposes. The exact type and version of these Linux instances are not important, but it must support installing specific packages to run protocols like LLDP and LACP. For this post, we used cirros-0.5.2.

1.4.3 Deploy vJunos-switch Instances

For this post, we will deploy the following fabric topology

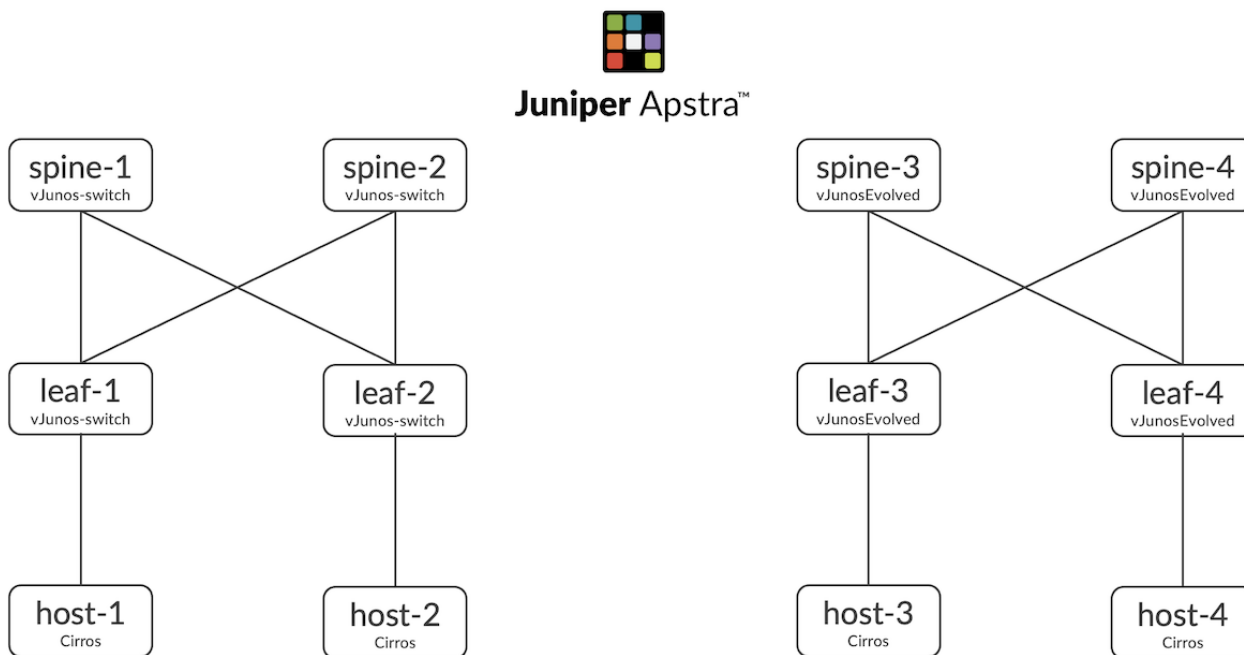


Figure 1: Network Topology

Given that the downloaded vJunos files are disk images (.qcow2) it is a best practice to make as many copies as there are vJunos instances to avoid attaching all vJunos instances to the same disk image. This will take up some disk space, so plan your setup accordingly. We will put these disk image copies under the default directory /var/lib/libvirt/images. The following bash script will help make these copies.

copy-images.sh:

```

1  #!/bin/bash
2  sudo cp cirros-0.5.2-x86_64-disk.img /var/lib/libvirt/images/host-1.img
3  sudo cp cirros-0.5.2-x86_64-disk.img /var/lib/libvirt/images/host-2.img
4  sudo cp cirros-0.5.2-x86_64-disk.img /var/lib/libvirt/images/host-3.img
5  sudo cp cirros-0.5.2-x86_64-disk.img /var/lib/libvirt/images/host-4.img
6  sudo cp vjunos-switch-23.1R1.8.qcow2 /var/lib/libvirt/images/vjunos-switch-23.1R1.8.qcow2
7  sudo cp vJunosEvolved-23.1R1.8-EV0.qcow2 /var/lib/libvirt/images/vJunosEvolved-23.1R1.8-EV0.qcow2
8  sudo qemu-img create -F qcow2 -b /var/lib/libvirt/images/vjunos-switch-23.1R1.8.qcow2 -f qcow2 /var/lib/libvirt/images/leaf-1.qcow2
9  sudo qemu-img create -F qcow2 -b /var/lib/libvirt/images/vjunos-switch-23.1R1.8.qcow2 -f qcow2 /var/lib/libvirt/images/spine-1.qcow2
10 sudo qemu-img create -F qcow2 -b /var/lib/libvirt/images/vjunos-switch-23.1R1.8.qcow2 -f qcow2 /var/lib/libvirt/images/leaf-2.qcow2
11 sudo qemu-img create -F qcow2 -b /var/lib/libvirt/images/vjunos-switch-23.1R1.8.qcow2 -f qcow2 /var/lib/libvirt/images/spine-2.qcow2
12 sudo qemu-img create -F qcow2 -b /var/lib/libvirt/images/vJunosEvolved-23.1R1.8-EV0.qcow2 -f qcow2 /var/lib/libvirt/images/leaf-3.qcow2

```

```

13 sudo qemu-img create -F qcow2 -b /var/lib/libvirt/images/vJunosEvolved-23.1R1.8-EV0.qcow2 -f
   qcow2 /var/lib/libvirt/images/spine-3.qcow2
14 sudo qemu-img create -F qcow2 -b /var/lib/libvirt/images/vJunosEvolved-23.1R1.8-EV0.qcow2 -f
   qcow2 /var/lib/libvirt/images/leaf-4.qcow2
15 sudo qemu-img create -F qcow2 -b /var/lib/libvirt/images/vJunosEvolved-23.1R1.8-EV0.qcow2 -f
   qcow2 /var/lib/libvirt/images/spine-4.qcow2

```

Using “qemu-img create” instead of a simple “cp” allows to have much smaller disk images, as shown below, which is good if you have a limited disk space.

```

1 user@host:~$ sudo ls -la /var/lib/libvirt/images/
2 total 25232500
3 drwx--x--x 2 root root 4096 Apr 5 16:00 .
4 drwxr-xr-x 7 root root 4096 Apr 3 23:42 ..
5 -rw-r--r-- 1 root root 4212916224 Mar 8 05:19 aos_server_4.1.2-269.qcow2
6 -rw-r--r-- 1 libvirt-qemu kvm 36306944 Apr 5 17:28 host-1.img
7 -rw-r--r-- 1 libvirt-qemu kvm 36306944 Apr 5 17:43 host-2.img
8 -rw-r--r-- 1 libvirt-qemu kvm 36306944 Apr 5 17:43 host-3.img
9 -rw-r--r-- 1 libvirt-qemu kvm 36306944 Apr 5 17:29 host-4.img
10 -rw-r--r-- 1 libvirt-qemu kvm 339017728 Apr 5 19:37 leaf-1.qcow2
11 -rw-r--r-- 1 libvirt-qemu kvm 338755584 Apr 5 19:37 leaf-2.qcow2
12 -rw-r--r-- 1 libvirt-qemu kvm 3691118592 Apr 5 19:37 leaf-3.qcow2
13 -rw-r--r-- 1 libvirt-qemu kvm 3619880960 Apr 5 19:37 leaf-4.qcow2
14 -rw-r--r-- 1 libvirt-qemu kvm 336592896 Apr 5 19:37 spine-1.qcow2
15 -rw-r--r-- 1 libvirt-qemu kvm 341442560 Apr 5 19:37 spine-2.qcow2
16 -rw-r--r-- 1 libvirt-qemu kvm 3619487744 Apr 5 19:37 spine-3.qcow2
17 -rw-r--r-- 1 libvirt-qemu kvm 3644981248 Apr 5 19:37 spine-4.qcow2
18 -rw-r--r-- 1 libvirt-qemu kvm 1766653952 Apr 5 16:00 vJunosEvolved-23.1R1.8-EV0.qcow2
19 -rw-r--r-- 1 libvirt-qemu kvm 3781951488 Apr 5 15:59 vjunos-switch-23.1R1.8.qcow2
20 user@host:~$

```

We will start by creating the networks of the above topology first then the create the virtual machines after, because this way, VMs can be effectively connected to the appropriate networks immediately when they boot up. It is our experience that when we create the VMs first, then the networks after, we need to reboot the VMs once more for connections to networks to take effect.

To deploy a network, we start by creating its xml definition file. The following is a sample network xml file.

leaf1-spine1.xml:

```

1 <network>
2   <name>leaf1-spine1</name>
3   <bridge stp='off' delay='0' />
4 </network>

```

then we execute the following commands to create a persistent network, and configure it to start automatically when libvirt daemon is restarted

```

1 user@host:~$ virsh net-define leaf1-spine1.xml
2 Network leaf1-spine1 defined from leaf1-spine1.xml
3
4 user@host:~$ virsh net-start leaf1-spine1
5 Network leaf1-spine1 started
6
7 user@host:~$ virsh net-autostart leaf1-spine1
8 Network leaf1-spine1 marked as autostarted
9
10 user@host:~$ virsh net-list
11 Name State Autostart Persistent
12 -----
13 default active yes yes
14 leaf1-spine1 active yes yes
15
16 user@host:~$

```

Note that we used “net-define” and not “net-create” because the former makes the network persistent, whereas the latter makes it transient, that is non persistent across reboots.

The reverse commands to delete the above network are the following:

```

1 user@host:~$ virsh net-destroy leaf1-spine1
2 Network leaf1-spine1 destroyed
3
4 user@host:~$ virsh net-undefine leaf1-spine1
5 Network leaf1-spine1 has been undefined
6
7 user@host:~$

```

The previous commands will preserve the xml definition file, so you can edit it again and start over.

We used the following bash script to create all the networks we need in this lab setup

create-networks.sh:

```

1 #!/bin/bash
2 echo "<network>" > macvtap.xml
3 echo " <name>macvtap</name>" >> macvtap.xml
4 echo " <forward mode='bridge'>" >> macvtap.xml
5 echo " <interface dev='ens9f0'>" >> macvtap.xml
6 echo " </forward>" >> macvtap.xml
7 echo "</network>" >> macvtap.xml
8 virsh net-define macvtap.xml
9 virsh net-start macvtap
10 virsh net-autostart macvtap
11 for network in leaf1-host1 leaf2-host2 leaf1-spine1 leaf1-spine2 leaf2-spine1 leaf2-spine2 \
12     leaf3-host3 leaf4-host4 leaf3-spine3 leaf3-spine4 leaf4-spine3 leaf4-spine4 \
13     leaf3_PFE_LINK leaf3_RPIO_LINK spine3_PFE_LINK spine3_RPIO_LINK leaf4_PFE_LINK
14     leaf4_RPIO_LINK spine4_PFE_LINK spine4_RPIO_LINK
15 do
16     echo "<network>" > $network.xml
17     echo " <name>$network</name>" >> $network.xml
18     echo " <bridge stp='off' delay='0'>" >> $network.xml
19     echo "</network>" >> $network.xml
20     virsh net-define $network.xml
21     virsh net-start $network
22     virsh net-autostart $network
23 done

```

The following bash script deletes all created networks with the previous script, in case you need to do so

delete-networks.sh:

```

1 #!/bin/bash
2 for network in leaf1-host1 leaf2-host2 leaf1-spine1 leaf1-spine2 leaf2-spine1 leaf2-spine2 \
3     leaf3-host3 leaf4-host4 leaf3-spine3 leaf3-spine4 leaf4-spine3 leaf4-spine4 \
4     leaf3_PFE_LINK leaf3_RPIO_LINK spine3_PFE_LINK spine3_RPIO_LINK leaf4_PFE_LINK
5     leaf4_RPIO_LINK spine4_PFE_LINK spine4_RPIO_LINK \
6     macvtap
7 do
8     virsh net-destroy $network
9     virsh net-undefine $network
10     rm $network.xml
11 done

```

After completing the previous task for all networks with your preferred method, you should see the following output

```

1 user@host:~$ virsh net-list
2 Name State Autostart Persistent
3 -----
4 default active yes yes
5 leaf1-host1 active yes yes
6 leaf1-spine1 active yes yes
7 leaf1-spine2 active yes yes
8 leaf2-host2 active yes yes
9 leaf2-spine1 active yes yes
10 leaf2-spine2 active yes yes
11 leaf3-host3 active yes yes
12 leaf3-spine3 active yes yes
13 leaf3-spine4 active yes yes
14 leaf3_PFE_LINK active yes yes
15 leaf3_RPIO_LINK active yes yes
16 leaf4-host4 active yes yes
17 leaf4-spine3 active yes yes
18 leaf4-spine4 active yes yes

```

```

19 leaf4_PFE_LINK active yes yes
20 leaf4_RPIO_LINK active yes yes
21 macvtap active yes yes
22 spine3_PFE_LINK active yes yes
23 spine3_RPIO_LINK active yes yes
24 spine4_PFE_LINK active yes yes
25 spine4_RPIO_LINK active yes yes
26
27 user@host:~$

```

1.4.4 Deploy vJunos

There are multiple ways to deploy vJunos instances on KVM, one can name at least these three:

- virt-manager : deploy all VMs by using KVM GUI
- virsh-define: this method requires creating an XML definition file of each VM
- virt-install : CLI based method. One needs only to specify the deployment parameters of VMs

virt-manager procedure works fine for small setups. The GUI is very intuitive, which makes this method less error prone. However, like most GUI-based tools, it does not scale well if you need to create a large number of VMs.

virsh-define is very error prone because the XML file is quite large and it is very easy to make syntax mistakes. It is highly recommended that you start with an existing XML file from a previously created and working VM, and not start from scratch, if you use virsh-define. If you do not have any XML file to start with, you can create a VM by using the virt-manager, then copy its XML file located under /etc/libvirt/qemu/, or generate it by using the command

```
1 virsh-dumpxml vm_name > vm_name.xml
```

Even though all 3 methods would work just fine, depending on the user's familiarity with each method. In this part we're going to use virt-install, because we can put all commands inside a shell script, and repeat the operation if anything is not working as expected. Like with any new product, it might take a few tries before you get everything right with all the parameters.

Below is a sample virt-install command to deploy one of the vJunos-switch VMs:

```

1 virt-install \
2   --name leaf-1 \
3   --vcpus 4 \
4   --ram 5120 \
5   --disk path=/var/lib/libvirt/images/leaf-1.qcow2,size=10 \
6   --os-variant generic \
7   --import \
8   --autostart \
9   --noautoconsole \
10  --nographics \
11  --serial ptty \
12  --cpu IvyBridge,+erms,+smep,+fsgsbase,+pdpe1gb,+rdrand,+f16c,+osxsave,+dca,+pcid,+pdc, +xtpr
13    ,+tm2,+est,+smx,+vmx,+ds_cpl,+monitor,+dtes64,+pbe,+tm,+ht,+ss,+acpi,+ds,+vme \
14  --sysinfo smbios,system_product=VM-VEX \
15  --network network=macvtap,model=virtio \
16  --network network=leaf1-spine1,model=virtio \
17  --network network=leaf1-spine2,model=virtio \
18  --network network=leaf1-host1,model=virtio

```

Below is a sample virt-install command to deploy one of the vJunosEvolved VMs:

```

1 virt-install \
2   --name leaf-3 \
3   --vcpus 4 \
4   --ram 5120 \
5   --disk path=/var/lib/libvirt/images/leaf-3.qcow2,size=10 \

```

```

6  --os-variant generic \
7  --import \
8  --autostart \
9  --noautoconsole \
10 --nographics \
11 --serial pty \
12 --cpu IvyBridge,+vmx \
13 --qemu-commandline="-smbios type=0,vendor=Bochs,version=Bochs -smbios type=3,manufacturer=
    Bochs -smbios type=1,manufacturer=Bochs,product=Bochs,serial=chassis_no=0:slot=0:type=1:
    assembly_id=0x0D20:platform=251:master=0:channelized=no" \
14 --network network=macvtap,model=virtio \
15 --network network=leaf3_PFE_LINK,model=virtio \
16 --network network=leaf3_RPIO_LINK,model=virtio \
17 --network network=leaf3_RPIO_LINK,model=virtio \
18 --network network=leaf3_PFE_LINK,model=virtio \
19 --network network=leaf3-spine3,model=virtio \
20 --network network=leaf3-spine4,model=virtio

```

The highlighted lines are important for a successful deployment of a vJunosEvolved instance.

Below is a sample virt-install command to deploy one of the Cirros VMs:

```

1  virt-install \
2  --name host-1 \
3  --vcpus 1 \
4  --ram 1024 \
5  --disk path=/var/lib/libvirt/images/host-1.img,size=10 \
6  --os-variant generic \
7  --import \
8  --autostart \
9  --noautoconsole \
10 --nographics \
11 --serial pty \
12 --network network=macvtap,model=virtio \
13 --network network=leaf1-host1,model=virtio

```

A few comments about the command above:

- `--serial pty` allows you to access the guest VM's console from the host by using `virsh console <vm_name>`. Another alternative to allow access to guest VMs from the host is via telnet by specifying, for example `--serial tcp,host=:4001,mode=bind,protocol=telnet`, where 4001 is a tcp port that is specific to the VM, so it must be different for each guest VM.
- The first interface of the VM is connected to `macvtap` bridge that connects to host's interface `eno1`. This way the VM will be connected to the same management network than the host itself, so we can reach it directly without jumping to the host. Other alternatives to how to manage the guest VM include the following:

`--network type=direct,source=eno1,source_mode=bridge,model=virtio`: this works the same way than `macvtap` bridge
`--network network=**default**,model=virtio`: this way, the VM will be connected to KVM's internal default network and will get an IP address via DHCP from the default 192.168.122.0/24 subnet. To make the VM reachable from outside, we need to configure the host for port forwarding.

We used a bash script to create the VMs needed in this lab setup. The script is not provided here for brevity, but it executes the above `virt-install` command for as many times as there are VMs with the specific parameters and interfaces.

Note that we provisioned each vJunos with 4 vCPU and 5 GB of RAM.

You can delete a VM by using the following commands, for example

```

1  virsh shutdown leaf-1
2  virsh undefine leaf-1
3  sudo rm /var/lib/libvirt/images/leaf-1.qcow2

```

Once all VMs are deployed, you should see the following output:

```

1 user@host:~$ virsh list
2   Id Name State
3   -----
4   13 leaf-1 running
5   14 leaf-2 running
6   15 leaf-3 running
7   16 leaf-4 running
8   17 spine-1 running
9   18 spine-2 running
10  19 spine-3 running
11  20 spine-4 running
12  21 host-1 running
13  22 host-2 running
14  23 host-3 running
15  24 host-4 running
16
17 user@host:~$

```

At this point, all VMs should be up and running, and should be reachable directly from the outside without jumping on the host. However, vJunos management interfaces run DHCP by default, so we do not know at this point what IP addresses have been assigned to vJunos instances. To that end, we must console to the instances either from the host by using “virsh console” or telnet to the specific port, as explained above, or by using virt-manager.

The vJunos instances should reach each other once we complete the basic Junos configurations. Let’s verify that.

1.4.5 Verifications

Try accessing the console of one of the vJunos-switch instances by using the following command

```

1 user@host:~$ virsh console leaf-1
2 Connected to domain leaf-1
3 Escape character is ^]
4
5
6 FreeBSD/amd64 (Amnesiac) (ttyu0)
7
8 login: root
9 Last login: Fri Mar 3 00:29:56 on ttyu0
10
11 --- JUNOS 23.1R1.8 Kernel 64-bit JNPR-12.1-20230203.cf1b350_buil
12 root@:~ #

```

The default credentials are “root” and no password.

Enable Junos CLI and verify that the dataplane is online

```

1 root@:~ # cli
2 root> show chassis fpc
3
4           Temp CPU Utilization (%) CPU Utilization (%) Memory Utilization (%)
5 Slot State (C) Total Interrupt 1min 5min 15min DRAM (MB) Heap Buffer
6  0 Online Testing 4 0 3 3 3 1023 19 0
7  1 Empty
8  2 Empty
9  3 Empty
10 4 Empty
11 5 Empty
12 6 Empty
13 7 Empty
14 8 Empty
15 9 Empty
16 10 Empty
17 11 Empty
18
19 root>

```

Verify that 10 “ge” interfaces are present

```

1 root> show interfaces ge* terse
2 Interface Admin Link Proto Local Remote
3 ge-0/0/0 up up
4 ge-0/0/0.16386 up up
5 ge-0/0/1 up up
6 ge-0/0/1.16386 up up
7 ge-0/0/2 up up
8 ge-0/0/2.16386 up up
9 ge-0/0/3 up down
10 ge-0/0/3.16386 up down
11 ge-0/0/4 up down
12 ge-0/0/4.16386 up down
13 ge-0/0/5 up down
14 ge-0/0/5.16386 up down
15 ge-0/0/6 up down
16 ge-0/0/6.16386 up down
17 ge-0/0/7 up down
18 ge-0/0/7.16386 up down
19 ge-0/0/8 up down
20 ge-0/0/8.16386 up down
21 ge-0/0/9 up down
22 ge-0/0/9.16386 up down
23
24 root>

```

A vJunos-switch instance comes up with 10 ge-x/x/x interfaces by default, but you can configure it with up to 96 ge-x/x/x interfaces by using the following command:

```
1 set chassis fpc 0 pic 0 number-of-ports 96
```

On the other hand, a vJunosEvolved instance comes up with 12 xe-x/x/x interfaces by default, and that cannot be changed by CLI.

Try accessing the console of one of the Cirros host VMs

```

1 user@host:~$ virsh console host-1
2 Connected to domain host-1
3 Escape character is ^]
4
5 login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
6 cirros login: cirros
7 Password:
8 $

```

If you used an Ubuntu image for the host VMs, you may not have access to the console with “virsh console”. If that’s the case, access the console with virt-manager and make the following changes in the guest VM:

```

1 sudo systemctl enable serial-getty@ttyS0.service
2 sudo systemctl start serial-getty@ttyS0.service

```

edit file `etc/default/grub` of the guest VM and configure the following lines

```

1 GRUB_CMDLINE_LINUX_DEFAULT="console=tty0 console=ttyS0"
2 GRUB_TERMINAL="serial console"

```

then

```
1 sudo update-grub
```

At this point, you should be able to access the Ubuntu VMs with “virsh console”.

Now that all looks good and we’re ready to start configuring our devices to build the EVPN-VXLAN topology shown above.

1.5 Configuration

1.5.1 Base vJunos-switch Configuration

When a vJunos-switch instance comes up, it has a default configuration that needs to be cleaned up before we proceed further. For example, the default configuration is ready for ZTP, which we will not address in this post.

The following configuration should be the bare minimum needed to start using a vJunos-switch instance, and you can delete everything else:

```
1 [edit]
2 root@leaf-1# show
3 ## Last changed: 2023-03-03 23:34:53 UTC
4 version 23.1R1.8;
5 system {
6     host-name leaf-1;
7     root-authentication {
8         encrypted-password "*****"; ## SECRET-DATA
9     }
10    services {
11        ssh {
12            root-login allow;
13        }
14        netconf {
15            ssh;
16        }
17    }
18    syslog {
19        file interactive-commands {
20            interactive-commands any;
21        }
22        file messages {
23            any notice;
24            authorization info;
25        }
26    }
27 }
28 interfaces {
29     fxp0 {
30         unit 0 {
31             family inet {
32                 dhcp {
33                     vendor-id Juniper-ex9214-VM64013DB545;
34                 }
35             }
36         }
37     }
38 }
39 protocols {
40     lldp {
41         interface all;
42         interface fxp0 {
43             disable;
44         }
45     }
46 }
47 [edit]
48 root@leaf-1#
```

A bare minimum vJunosEvolved configuration is very similar, except the management interface name, which is re0:mgmt-0 for vJunosEvolved instead of fxp0 for vJunos-switch.

Once all vJunos-switch instances have the minimum configuration above entered and committed, let us verify that the topology is built properly by verifying LLDP neighborship.

```
1 [edit]
2 root@leaf-1# run show lldp neighbors
3
4 [edit]
5 root@leaf-1#
```

```

6
7 [edit]
8 root@leaf-1# run show lldp statistics
9 Interface Parent Interface Received Unknown TLVs With Errors Discarded TLVs Transmitted
   Untransmitted
10 ge-0/0/0 - 0 0 0 0 127 0
11 ge-0/0/1 - 0 0 0 0 127 0
12 ge-0/0/2 - 0 0 0 0 127 0
13
14 [edit]
15 root@leaf-1#

```

The output above shows that LLDP neighborship are not forming, and that the instance is transmitting LLDP packets but is not receiving any. This is expected because, by default, IEEE 802.1D compliant bridges Linux bridges do not forward frames of link local protocols, like LLDP and LACP. For reference, the destination MAC address used by LLDP is 01-80-C2-00-00-0E.

Let's checking the default value of the Group Forwarding Mask of one of the bridges

```

1 user@host:~$ cat /sys/class/net/virbr1/bridge/group_fwd_mask
2 0x0
3 user@host:~$

```

The zero value means that this bridge does not forward any link local protocol frames. To change the bridge's behavior and force it to forward LLDP frames we need to set the 15th bits of this 16-bit mask, specified by the rightmost 0xE in the MAC address, that is writing the hex value 0x4000, or decimal value $2^{14}=16,384$, to group_fwd_mask. Let's try it on all bridges of this host.

We used the following shell script to accomplish that. Please adjust bridge and interface numbers to your specific case ; use "brctl show" command to get the number of virbrX bridges and vnetX interfaces

```

1 user@host:~$ brctl show
2 bridge name bridge id STP enabled interfaces
3 virbr0 8000.5254006f5abc yes virbr0-nic
4 virbr1 8000.525400e46740 no virbr1-nic
5
6 vnet2
7 virbr10 8000.5254004bf7c3 no virbr10-nic vnet34
8
9 vnet11
10 virbr11 8000.525400158ad5 no virbr11-nic vnet32
11
12 vnet16
13 virbr12 8000.525400a700c7 no virbr12-nic vnet27
14
15 vnet17
16 virbr13 8000.525400b539f0 no virbr13-nic vnet33
17
18 vnet6
19 virbr14 8000.5254007070dc no virbr14-nic vnet9
20
21 vnet7
22 virbr15 8000.5254005ed09f no virbr15-nic vnet8
23
24 vnet22
25 virbr16 8000.52540012d521 no virbr16-nic vnet25
26
27 vnet23
28 virbr17 8000.525400895359 no virbr17-nic vnet24
29
30 vnet12
31 virbr18 8000.5254007bc6c4 no virbr18-nic vnet15
32
33 vnet13
34 virbr19 8000.5254003dffb0 no virbr19-nic vnet14
35
36 vnet28
37 virbr2 8000.5254007a6662 no virbr2-nic vnet31
38
39 vnet35
40 virbr20 8000.52540062fca2 no virbr20-nic vnet5
41
42 vnet29

```

```

42
43 virbr3 8000.525400d2ebec no virbr3-nic vnet30
44 vnet0
45 vnet18
46 virbr4 8000.52540036dab2 no virbr4-nic
47 vnet1
48 vnet20
49 virbr5 8000.525400831b16 no virbr5-nic
50 vnet19
51 vnet3
52 virbr6 8000.525400a74b01 no virbr6-nic
53 vnet21
54 vnet4
55 virbr7 8000.52540010f358 no virbr7-nic
56 vnet36
57 virbr8 8000.525400587638 no virbr8-nic
58 vnet37
59 virbr9 8000.525400434614 no virbr9-nic
60 vnet10
61 vnet26
62 user@host:~$

```

overwrite-mask.sh:

```

1 #!/bin/bash
2 for i in {0..20}
3 do
4     echo 0x4000 > /sys/class/net/virbr$i/bridge/group_fwd_mask
5 done

```

At this point, LLDP should be working fine on all vJunos-switch instances, as shown below

```

1 [edit]
2 root@leaf-1# run show lldp statistics
3 Interface Parent Interface Received Unknown TLVs With Errors Discarded TLVs Transmitted
  Untransmitted
4 ge-0/0/0 - 10 0 0 0 170 0
5 ge-0/0/1 - 10 0 0 0 169 0
6 ge-0/0/2 - 0 0 0 0 167 0
7
8 [edit]
9 root@leaf-1# run show lldp neighbors
10 Local Interface Parent Interface Chassis Id Port info System Name
11 ge-0/0/0 - 2c:6b:f5:19:f5:c0 ge-0/0/0 spine-1
12 ge-0/0/1 - 2c:6b:f5:a6:fc:c0 ge-0/0/0 spine-2
13
14 [edit]
15
16 root@leaf-1# run show lldp neighbors
17 Local Interface Parent Interface Chassis Id Port info System Name
18 ge-0/0/0 - 2c:6b:f5:1a:65:c0 ge-0/0/0 spine-1
19 ge-0/0/1 - 48:cd:91:2e:05:d5 et-0/0/0 spine-2
20
21 [edit]
22 root@leaf-1#

```

Note that LLDP neighborship is not forming between leaf-1 and host-1, and that's because Linux does not include LLDP package by default. It can be installed on some Linux distributions, but not on Cirros, that we used here.

1.6 Adding Apstra Controller

Now that all vJunos instances are deployed and working properly, we will onboard them on Apstra server and deploy the fabric.

Please note the following important requirements:

- You need Juniper Apstra version 4.1.1 or higher to manage vJunos instances ; at the time of writing this post, we used Juniper Apstra version 4.1.2-269

- By default, vJunos-switch comes up with 10 GbE interfaces, but you can configure it for up to 96 GbE interfaces. Also, Juniper Apstra has a default Device Profile for vJunos-switch, called vEX, that has 10 1GbE/10GbE interfaces, and this DP is automatically associated with all vJunos-switch instances. If your topology uses 10 or less dataplane interfaces, all should work just fine. However, if your topology requires more than 10 interfaces, then additional configuration both on vJunos-switch instances and Juniper Apstra will be required.
- vJunosEvolved instance is a virtual representation of the Juniper PTX10001-36MR, as shown below, and therefore we could just use the Device Profile of that platform that comes bundled with Juniper Apstra 4.1.2. However, because vJunos-switch and vJunosEvolved versions that we used in this lab, are release 23.1R1, we needed to create new Device Profiles to make the change necessary change in the Selector of the Device Profiles to include this version. We changed the RegEx from (1[89]|2[0-3])..* to (1[89]|2[0-3])..* for vJunos-switch and from (20.[34].|2[12]..)-EVO\$ to (20.[34].|2[123]..)-EVO\$ for vJunosEvolved.

```
1 root@leaf-1> show system information
2 Model: ex9214
3 Family: junos
4 Junos: 23.1R1.8
5 Hostname: leaf-1
6
7 root@leaf-1>
```

And,

```
1 root@spine-3> show system information
2 Model: ptx10001-36mr
3 Family: junos
4 Junos: 23.1R1.8-EVO
5 Hostname: spine-3
6
7 root@spine-3>
```

In our setup, leaf-1, leaf-2, spine-1 and spine-2 are running vJunos-switch and leaf-3, leaf-4, spine-3 and spine-4 are running vJunosEvolved, so we will create 2 separate interface maps and rack types, then create a template and a blueprint that includes both racks to build two 2 x leaf, 2-spine EVPN-VXLAN fabrics. The fabrics will have routing on the leafs, in ERB style, and we will show that everything is working fine by testing connectivity between host-1 in subnet 10.1.1.0/24 and host-2 in subnet 10.1.2.0/24 on one fabric, and between host-3 in subnet 10.2.1.0/24 and host-4 in subnet 10.2.2.0/24 on the other fabric.

Note that because vJunos-switch simulates an EX9214 system with redundant Routine-Engine, you must add the following CLI commands to the pristine configuration before onboarding leaf-1 and spine-1 on Apstra

```
1 set system commit synchronize
2 set chassis evpn-vxlan-default-switch-support
```

The other caveat to be aware of, is that vJunosEvolved leafs need the following command to be configured, so you need to push this configuration via an Apstra configlet

```
1 set forwarding-options tunnel-termination
```

Apstra configuration steps for vJunos instances are no different than for all other hardware platforms so we will not share details here for the sake of brevity.

1.7 Conclusion

In this post, we shared the step to successfully deploy vJunos-switch and vJunosEvolved virtual appliances. Our purpose was to provide all the details to avoid running into issues that might cause long

troubleshooting sessions.

We hope we achieved this goal.

1.8 Useful links

- vJunos-switch: <https://support.juniper.net/support/downloads/?p=vjunos>
- vJunosEvolved: <https://support.juniper.net/support/downloads/?p=vjunos-evolved>
- Juniper Apstra documentation: <https://www.juniper.net/documentation/product/us/en/apstra/>
- Deploying and Using Junos in a Bare Metal EVEN-NG Server: <https://juniper.github.io/techposts/deploying-vjunos-in-a-bare-metal-eve-ng-server/article>

1.9 Glossary

- DHCP : Dynamic Host Configuration Protocol
- ERB : Edge Routing and Bridging
- LACP : Link Aggregation Control Protocol
- LLDP : Link Layer Discovery Protocol

1.10 Acknowledgements

Special thanks to the following individuals who helped understand implementation details of vJunos-switch and vJunosEvolved, to those who helped in building and troubleshooting the setups we used in this post, and who also helped reviewing this post:

- Aninda Chatterjee
- Art Stine
- Hartmut Schroeder
- Kaveh Moezzi
- Shalini Mukherjee
- Yogesh Kumar
- Vignesh Shanmugaraju



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.

Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com

